Back To Basics
Containers
Part 1: Classic Containers

ALAN TALBOT



Cppcon
The C++ Conference

20
26



September 12 - 18

Introduction

- I will be here at the end for questions
  - If we don’t get to yours, post them to the **back-to-basics-containers** discord channel
  - Or look for me in the hall!
- This talk is about the classic C++ Standard Library containers and adaptors
  - C++98 and C++11
- Alan Talbot
  - Using C++ almost exclusively for over 35 years
  - On the C++ committee (WG21) for 21 years

Why Talk About Containers?

- Containers are a ubiquitous and important feature of the Standard Library
- Container choice has a big impact on performance
  - The reason we have so many containers is to provide different performance profiles
- How containers are used also can have a big impact on performance
- A solid understanding of containers is not so ubiquitous
  - The container design provides guidance based on some outdated premises
- Performance these days is greatly dependent on proper cache management
  - Accessing main memory can be 50-100 times slower than L1 cache
  - Performance depends on locality *and* predictability of reference
  - Reusing a memory allocation is much faster than deallocating and reallocating

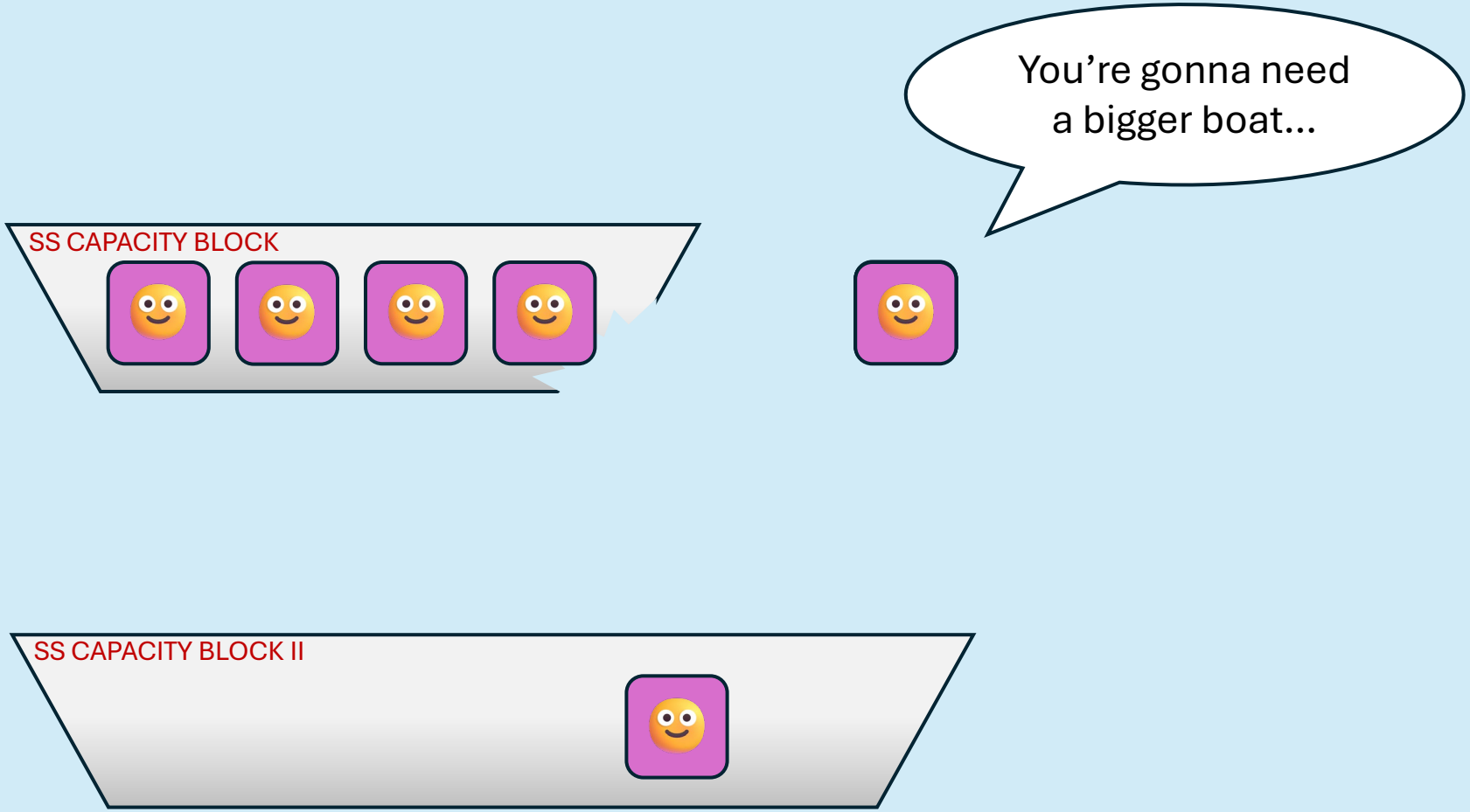
Contiguous Containers

- Block-based containers with only one memory block
  - The block may be dynamically allocated on the heap
  - Or local (contained within the container object)
- The elements in the memory block are adjacent
  - So they have optimal iteration semantics
  - And optimal locality/predictability of reference
  - Yielding the best possible iteration performance
- **vector**
  - This is a dynamic sequence container (up next)
- **inplace_vector**
  - This is a local sequence container (discussed in Part 2)
- **array** is considered a container, but it’s an aggregate

vector

- Elements are kept at the top of the block
  - Empty space is after the last element
- *O*(1) insertion is offered at the back if there is room in the block
- If the block is full on insertion, a new, larger block is allocated
  - The growth is geometric
  - The factor is implementation defined (typically 1.5 or 2)
  - All existing elements are moved to the new block
  - All links (iterators, references and pointers) are invalidated
- Deleting elements or calling **clear** does not deallocate the block
- The block is deallocated when the **vector** is destructed
  - Or on calling **shrink_to_fit** if the **vector** is empty

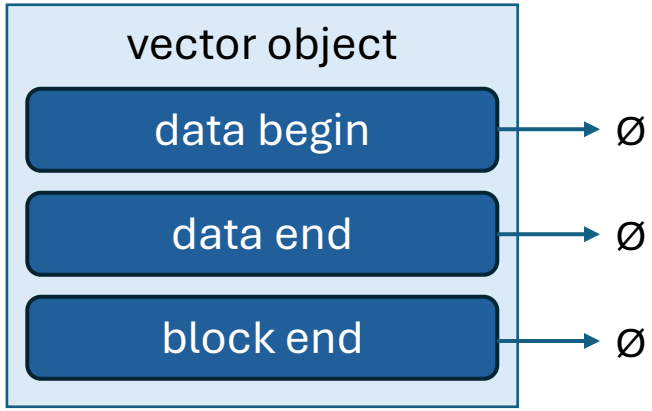
vector


You’re gonna need
a bigger boat...
SS CAPACITY BLOCK
SS CAPACITY BLOCK II

vector

- When you run out of room, you need a lot of available space
  - The new block is either 50% or 100% larger than the previous block
  - Temporarily both the new and old blocks have to exist at once
  - This leads to memory fragmentation as the **vector** grows
- So there is a limit to how big a **vector** can grow
  - Theoretically the limit is 3/5 (for 1.5) or 2/3 (for 2.0) of available memory
  - In practice it will be much less due to fragmentation and not hitting the numbers exactly
- Growing a vector is also expensive time-wise
  - Each growth is an allocation and a deallocation
  - All the elements are moved/copied (which will dominate for large sizes)
- You can **reserve** a predetermined capacity to avoid or limit growth thrashing

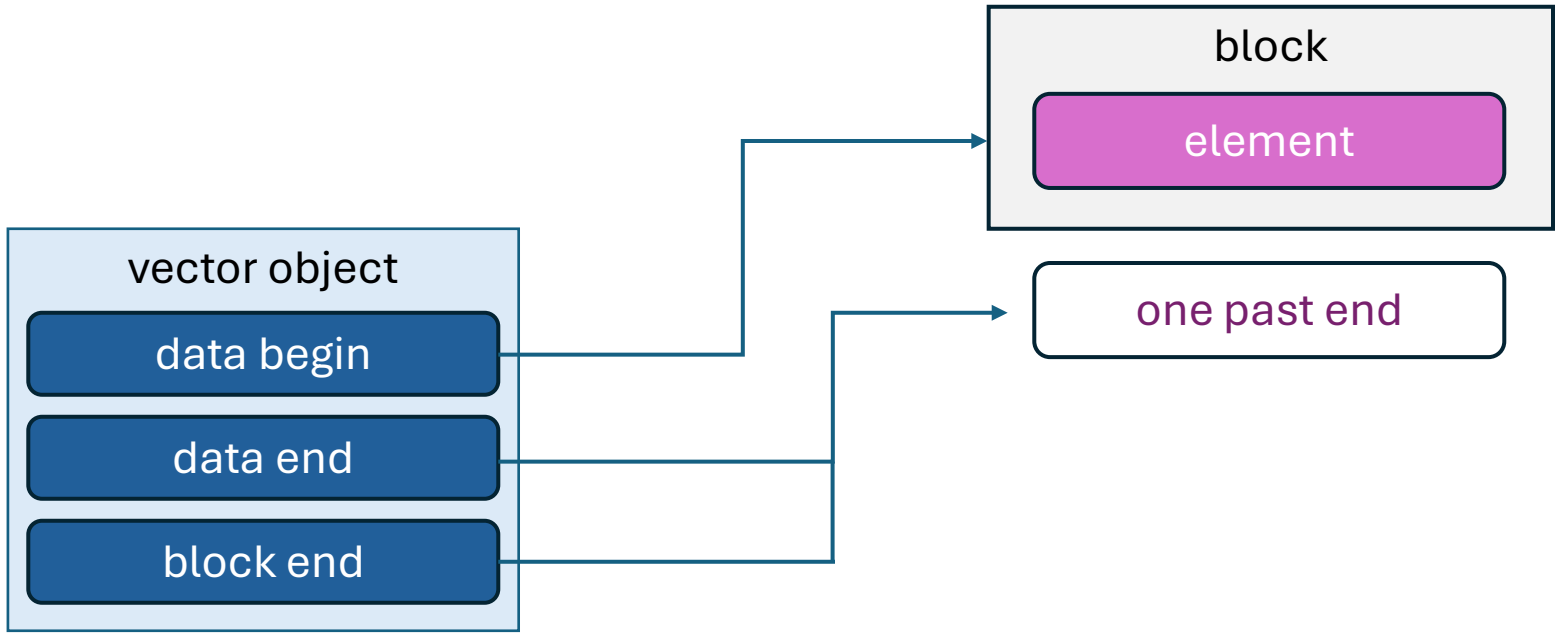
vector – Default Constructed



size = 0

capacity = 0

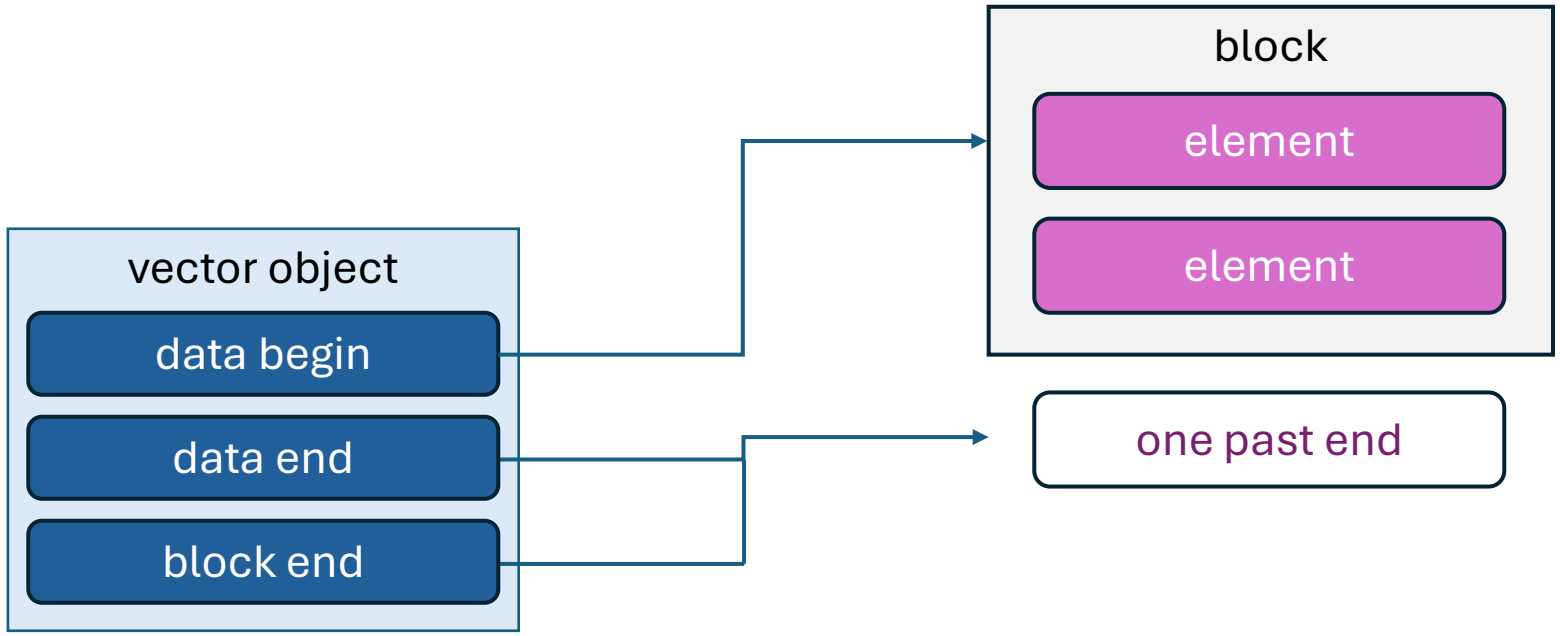
vector – Growth



size = 1

capacity = 1

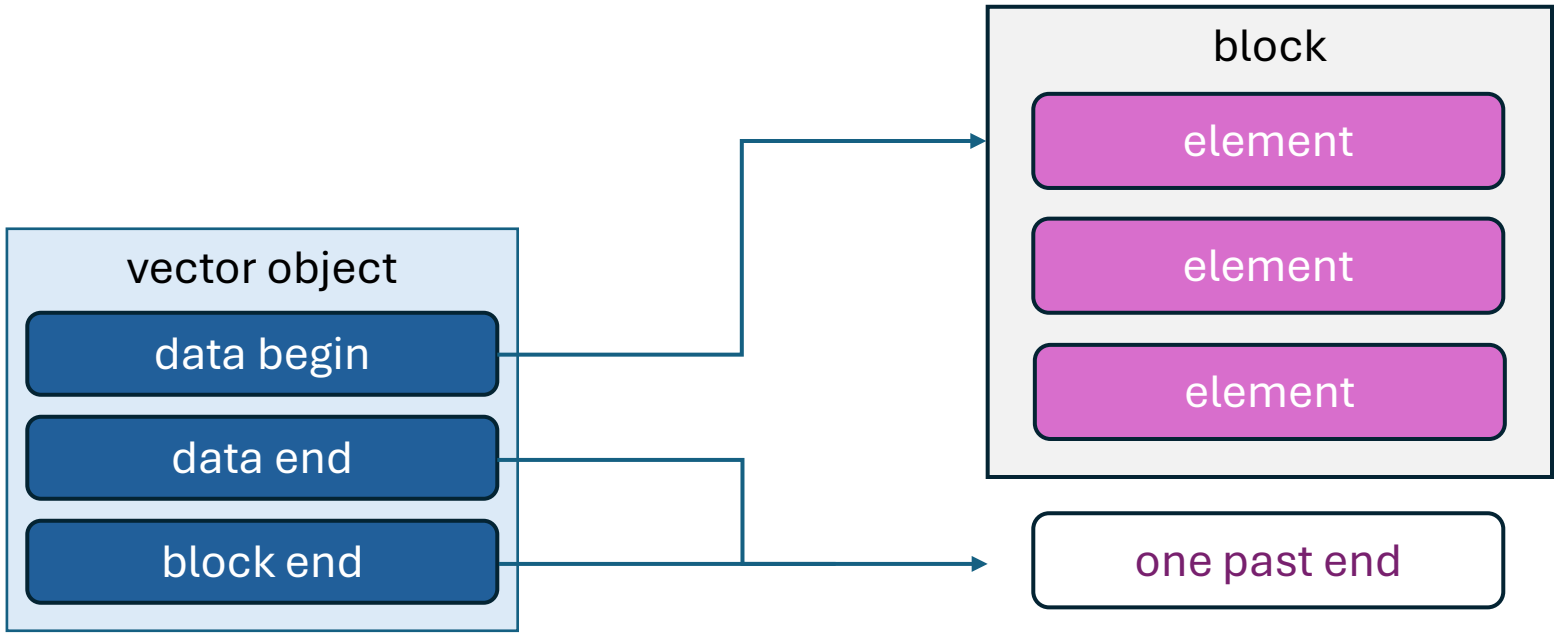
vector – Growth



size = 2

capacity = 2

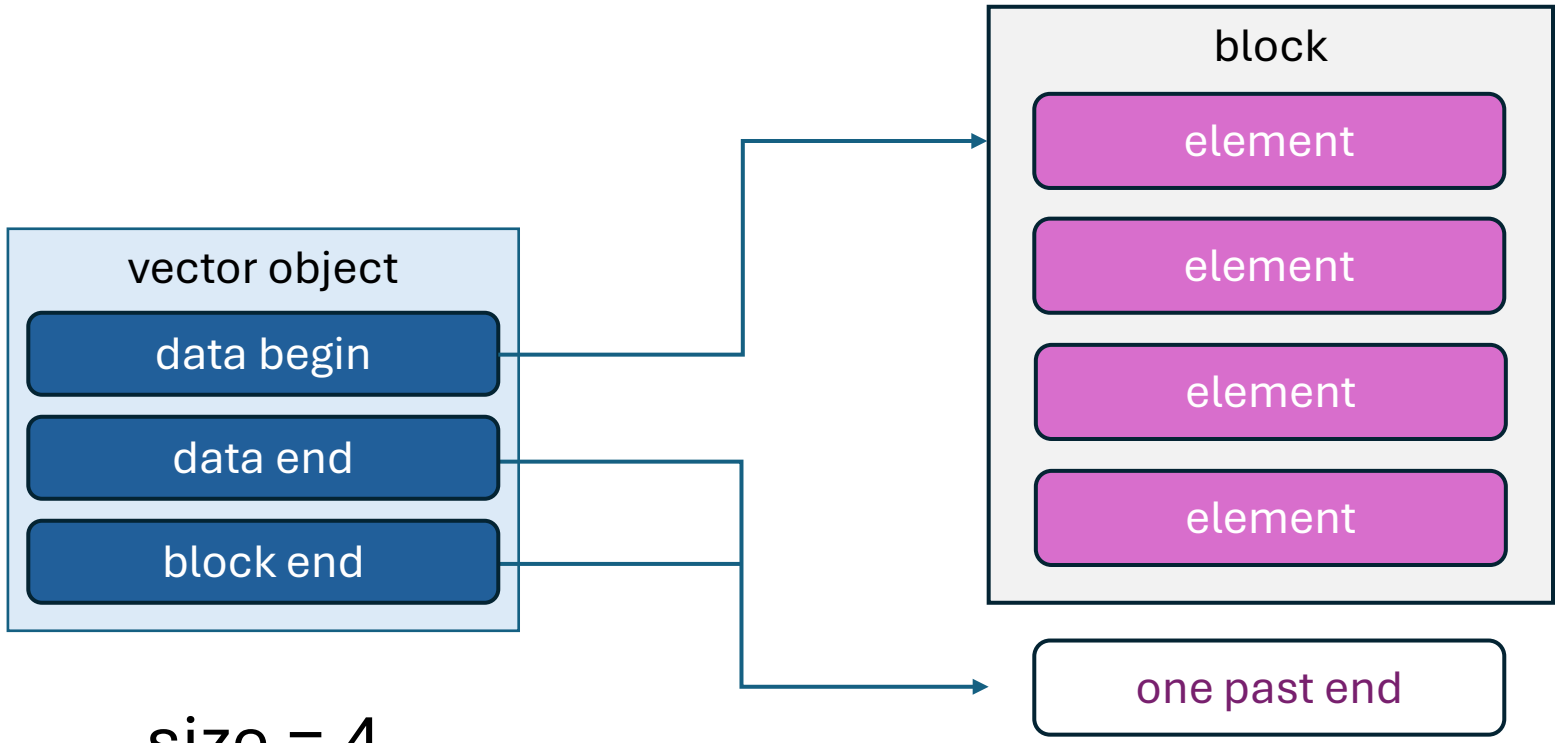
vector – Growth



size = 3

capacity = 3

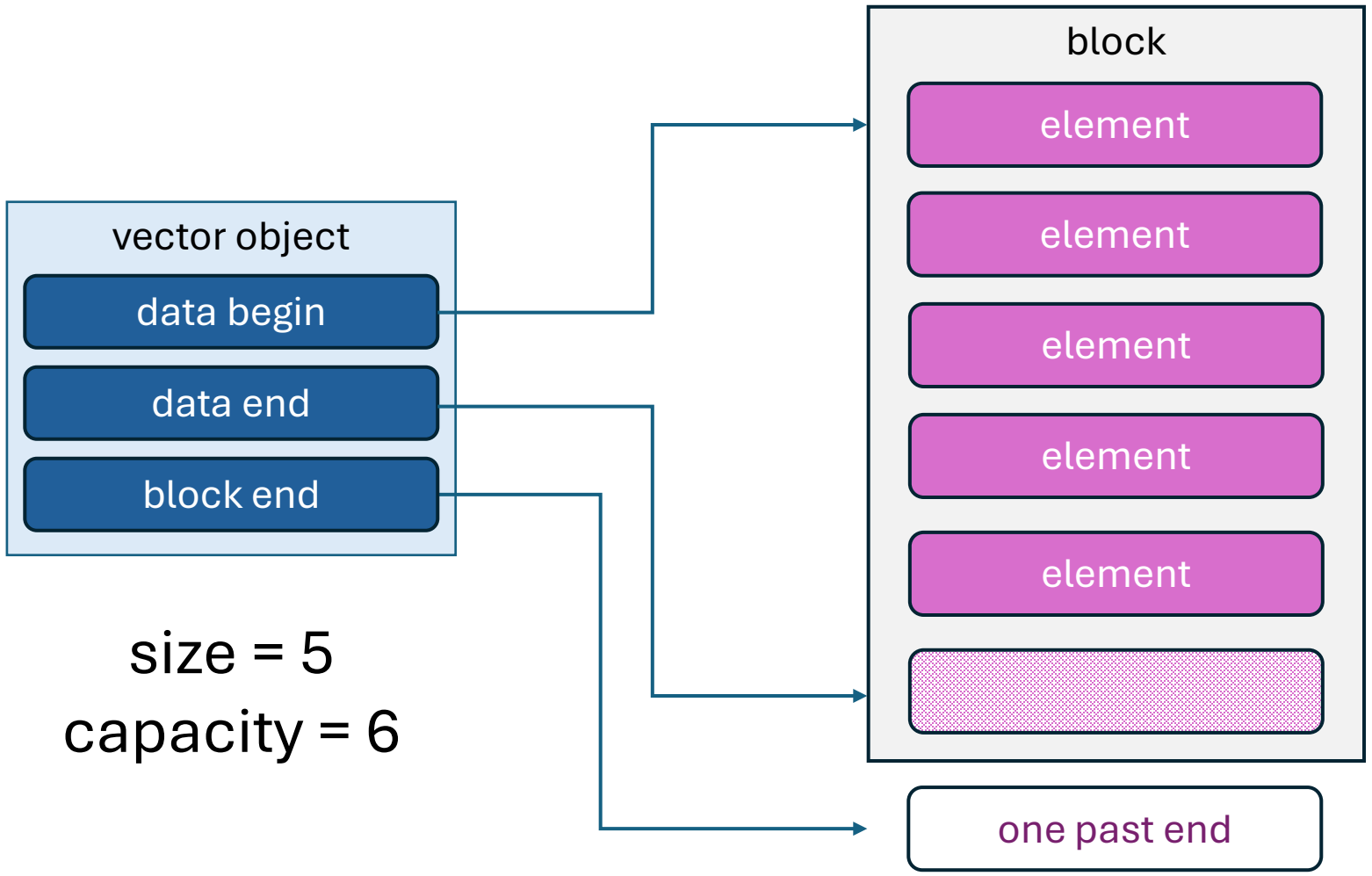
vector – Growth



size = 4

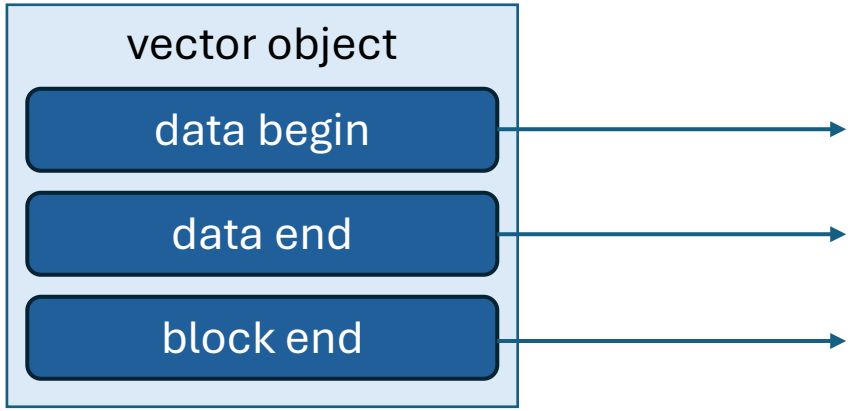
capacity = 4

vector – Growth



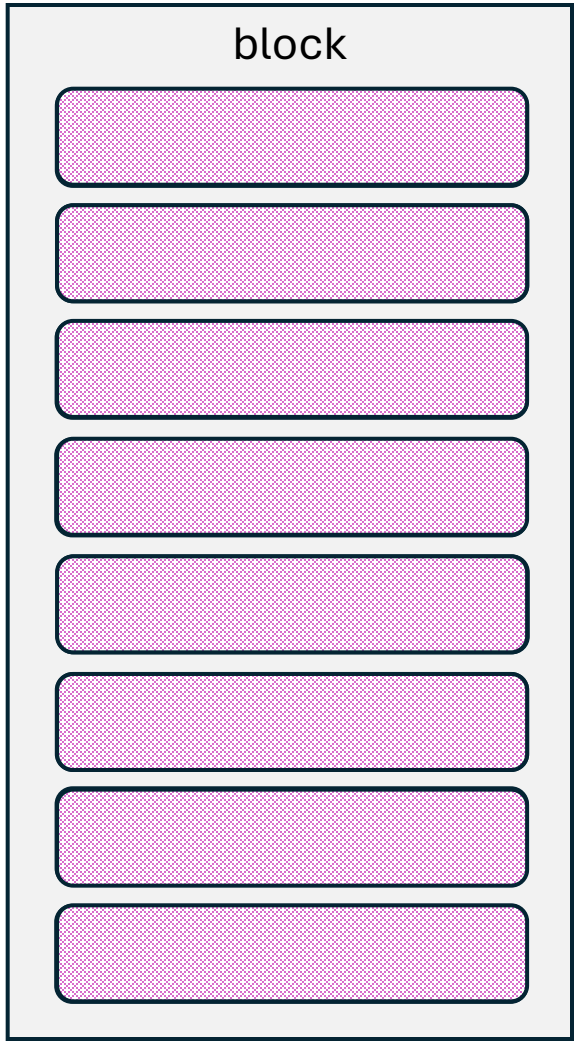
vector – Reserve

reserve(8)



size = 0

capacity = 8



vector – Capacity History

- Because of the way vector grows, it develops a capacity history
  - After a while, the capacity will be correct for the job
  - You may not know what to reserve ahead of time
  - If you clear the vector, the capacity history is preserved
  - If you destroy the vector the capacity history is lost
- What if you construct a vector inside a loop?
  - You are destroying the vector every time around
  - So you lose the history and have to climb back up to the high-water mark
  - And your new memory will not be in cache
- If you are using many vectors at once
  - The fragmentation is worse
  - And the problem is much harder to spot in the code

vector Example – The Problem

- I unknowingly created just this situation in a railroad simulator
- The simulation was using a bunch of temporary **vector**s
- The run time increased by 100 when the number of time steps increased by 10
- Profiling indicated that lots of time was being spent in **vector** allocation
  - But I thought I was doing everything right!
  - I used thoughtful pre-reserving
  - I was careful to use move semantics
- We can see the problem
  - I was thrashing the heap with **vector** construction/destruction
  - I was not preserving the capacity history

vector Example – Solutions

- Hoisting the **vector** declaration out of the loop
  - This keeps the vector around and preserves the capacity history
  - It works well if there is one **vector** and the control flow is simple
  - But it's not practical for more complicated situations
- Recycling
  - Expired **vector**s can be moved into a container, which preserves the block
  - This works well if you have many vectors in play at once
  - Or if you need access across the program
- I used the recycling technique in the railroad simulator
  - The run time of a particular simulation went from 7:15 to 0:35 (H:M)
  - This was a huge improvement: a speed up of 12 times
  - The customers were very happy
  - And it was very simple to implement

vector Example – Code

```
class number_list {
public:
    number_list()
    {
        if (recycling.empty())
            numbers.reserve(TUNED_GUESS);
        else
        {
            numbers = move(recycling.back());
            recycling.pop_back();
        }
    }
    ~number_list()
    {
        numbers.clear();
        recycling.push_back(move(numbers));
    }
    static void empty_recycling() { recycling.clear(); }

    using container_type = vector<int>;
    container_type numbers;

private:
    static vector<container_type> recycling;
};
```

vector<bool>

- The most (in)famous blunder in the Standard Library
  - Introduced in the original STL and evoking hilarity (or something) ever since
- A recent research paper used **vector<bool>** expecting… a vector of **bool**s
  - (how droll)
- The paper compared performance across several languages and guess what?
  - C++ failed big
  - Which had nothing to do with C++ (the language)
  - And everything to do with **std::vector<bool>**
- And the paper authors can’t be blamed for making this mistake
  - Why would a non-C++ expert think that the only type in all the world that causes **vector** not to do what you want is **bool**?

vector<bool>

- **vector<bool>** is not a container because it can't meet the requirements
  - There are no bit references in C++, so it can't return one
- And **vector<bool>** is a misnomer: it's *not* a vector of **bool**
  - A vector of byte-sized values can be had with **vector<uint8_t>** (or **char** or whatever)
- It's actually a bitset, which is a very useful data structure
  - The SL has a static-size bitset called (remarkably) **bitset**
  - Boost has a dynamically-sized bitset called **dynamic_bitset**
- But **vector<bool>** is not a very good bitset
  - In most libraries it has no specialized algorithms
  - But the standard algorithms access bits individually, which is a big pessimization
  - And its **vector** API is ill-fitted to the task
- Just say no—because you probably won't *ever* get what you want

Block-based Containers

- Block-based containers that can have *more* than one memory block
  - The blocks are dynamically allocated on the heap
- Emptied blocks may or may not be deallocated
  - This is implementation defined
- **deque**
  - This is a sequence container (up next)
- **hive**
  - This is not quite a sequence container (discussed in Part 2)

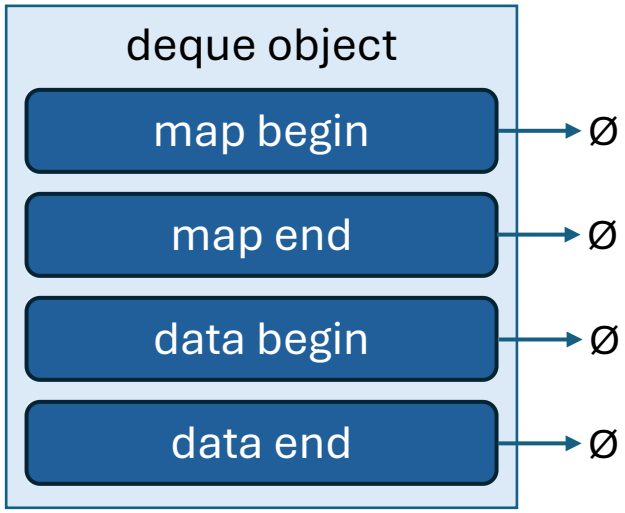
deque

- **deque** is short for double-ended queue
  - But while **std::deque** does implement a double-ended queue, it's much more than that
- **deque** keeps the elements in dynamically allocated blocks
- The blocks are managed by a dynamically allocated "map"
  - An array of pointers to blocks, not a **std::map**
- *O*(1) insertion at both the front and back, if there is room in the block
- If there is no room for an insertion, a new block is allocated
  - Block size is fixed and implementation defined
  - For front or back insertion no existing elements are moved (so links are not invalidated)

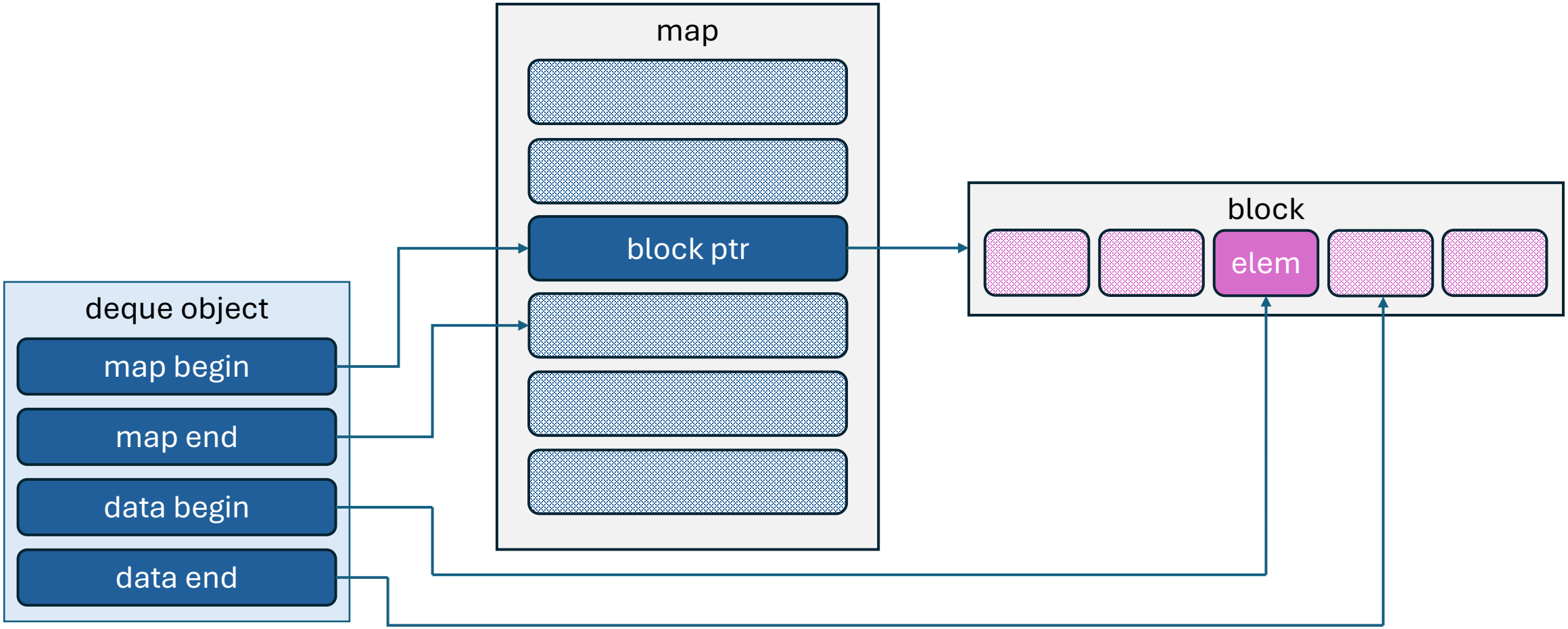
deque

- **deque** offers most of the performance of **vector**
  - The elements in each memory block are adjacent
  - It has random access iterators
  - Within the block, the locality/predictability of reference is optimal
  - Yielding very good iteration performance
- **deque** offers fine-grained linear memory growth
  - Unlike the geometric growth of **vector**
  - This lets you fill up memory much more completely
  - And growth will not cause fragmentation (no deallocations)
  - This is a major reason to use **deque**
- Unfortunately the block size is not user-controllable
  - So you can’t tune your performance
  - And some libraries have absurdly small block sizes (see Links)

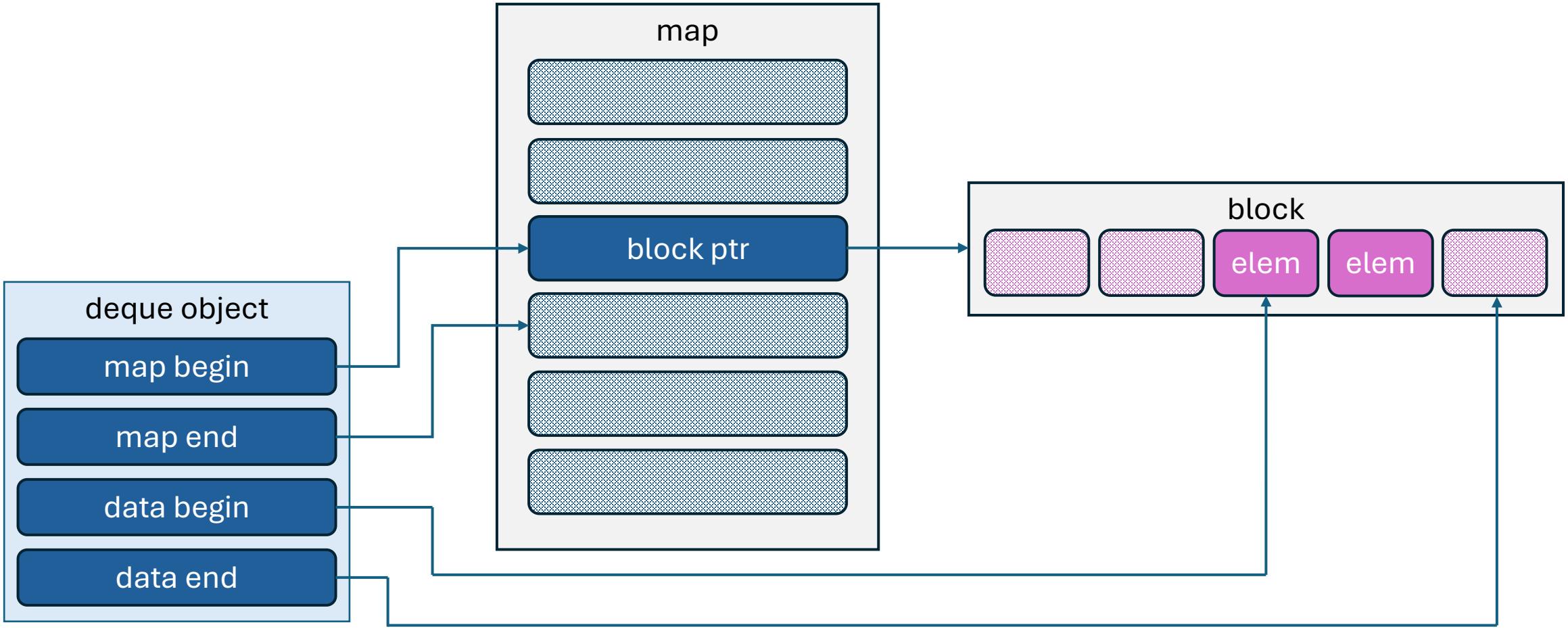
deque – Default Constructed



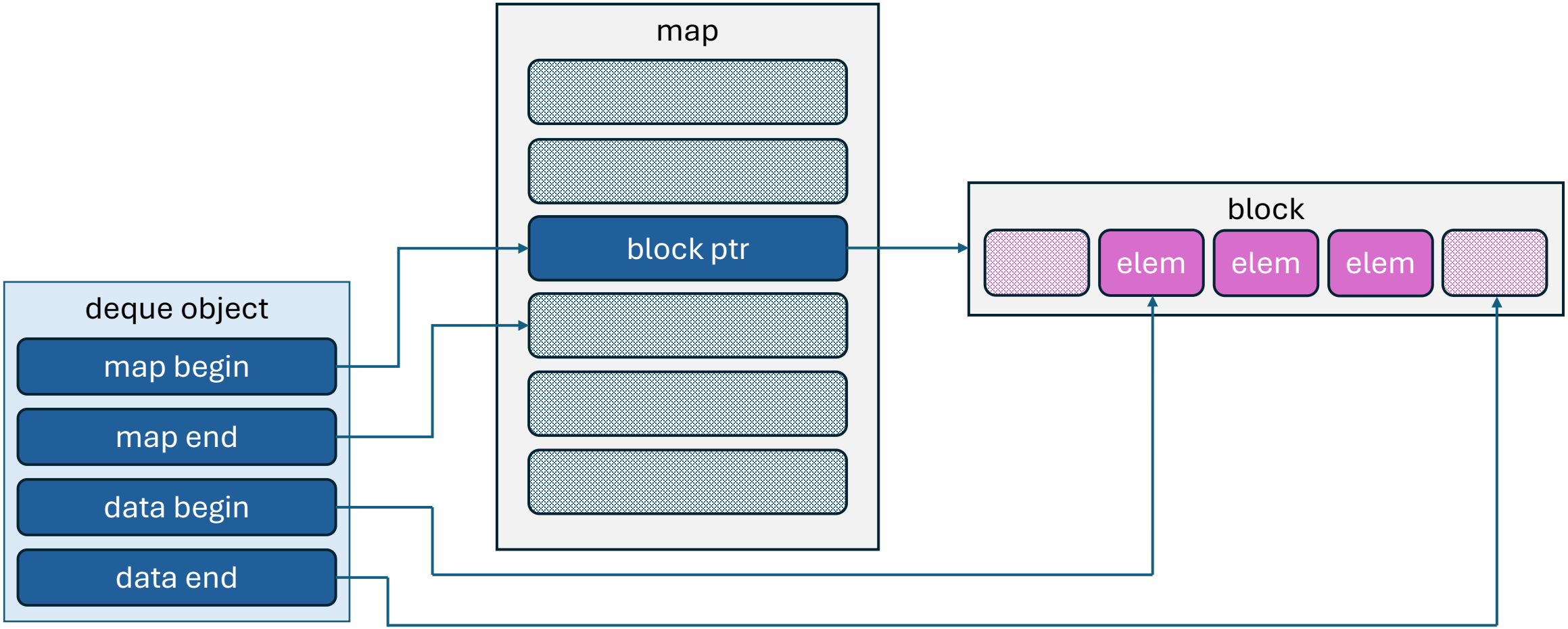
deque – First Block Created



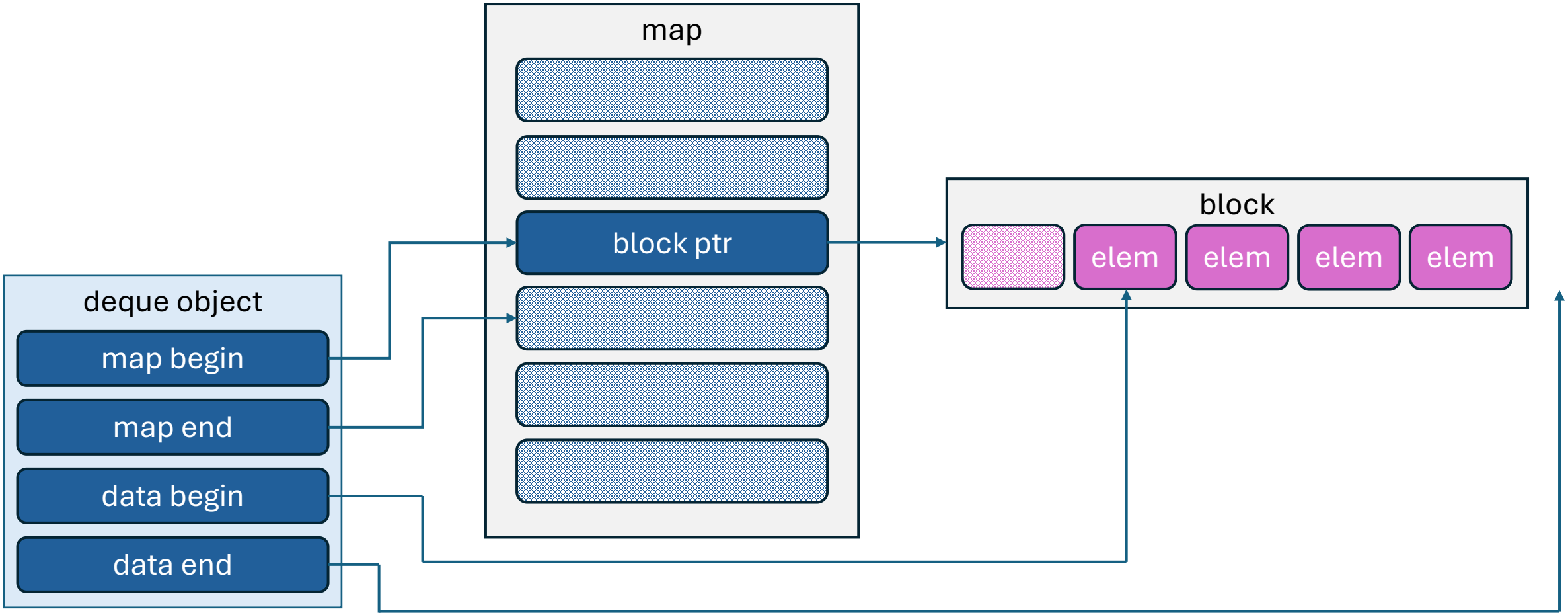
deque – First Block Created



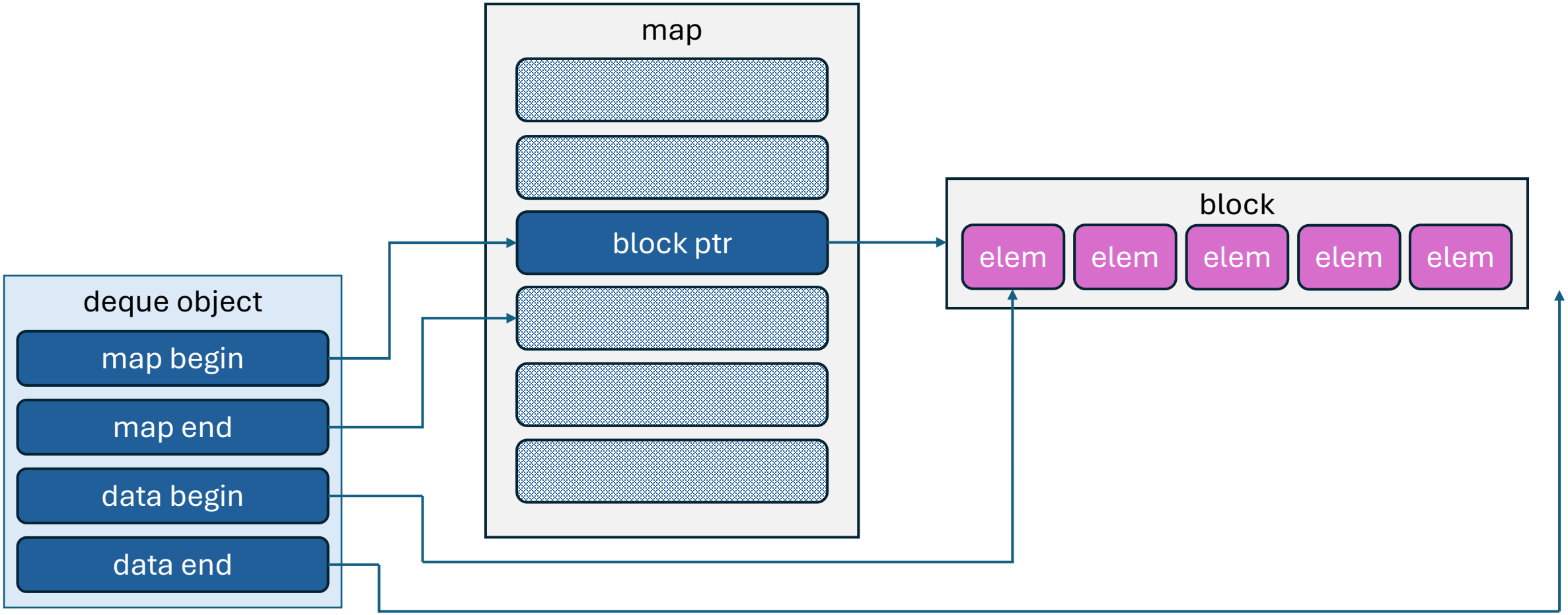
deque – First Block Created



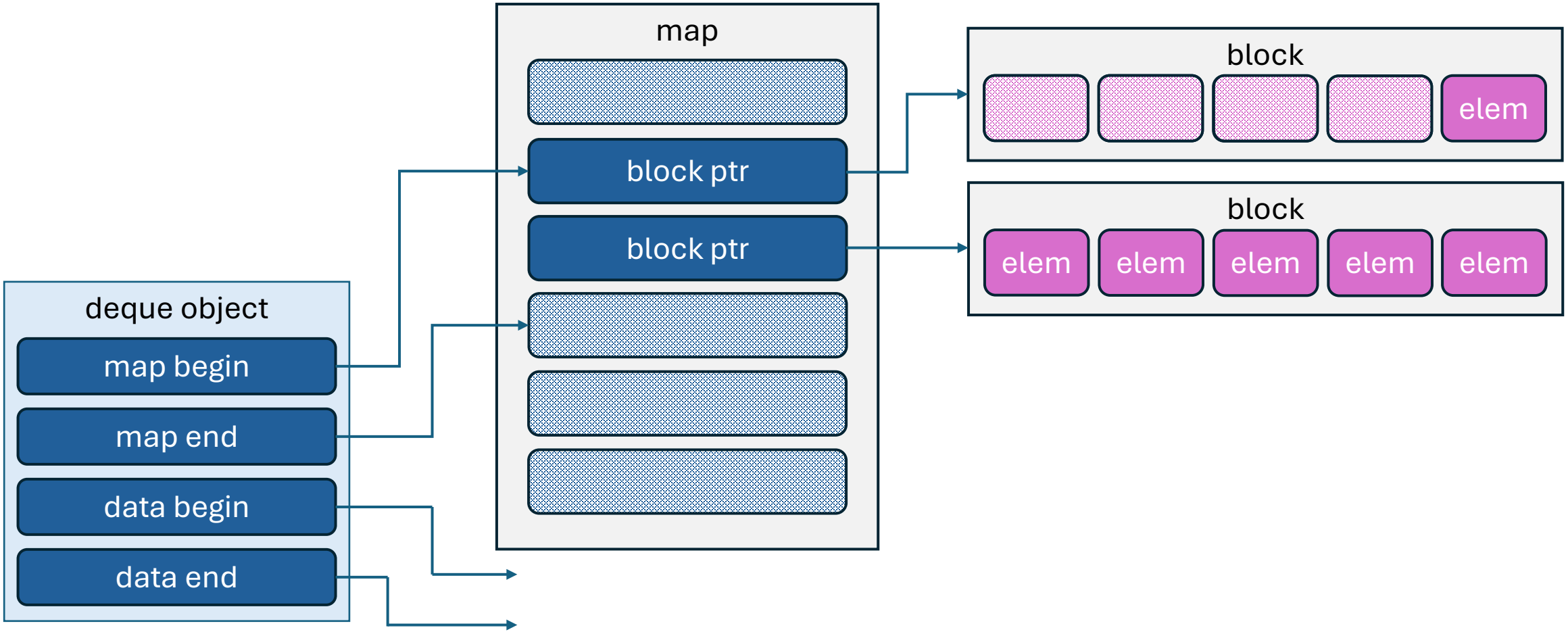
deque – First Block Created



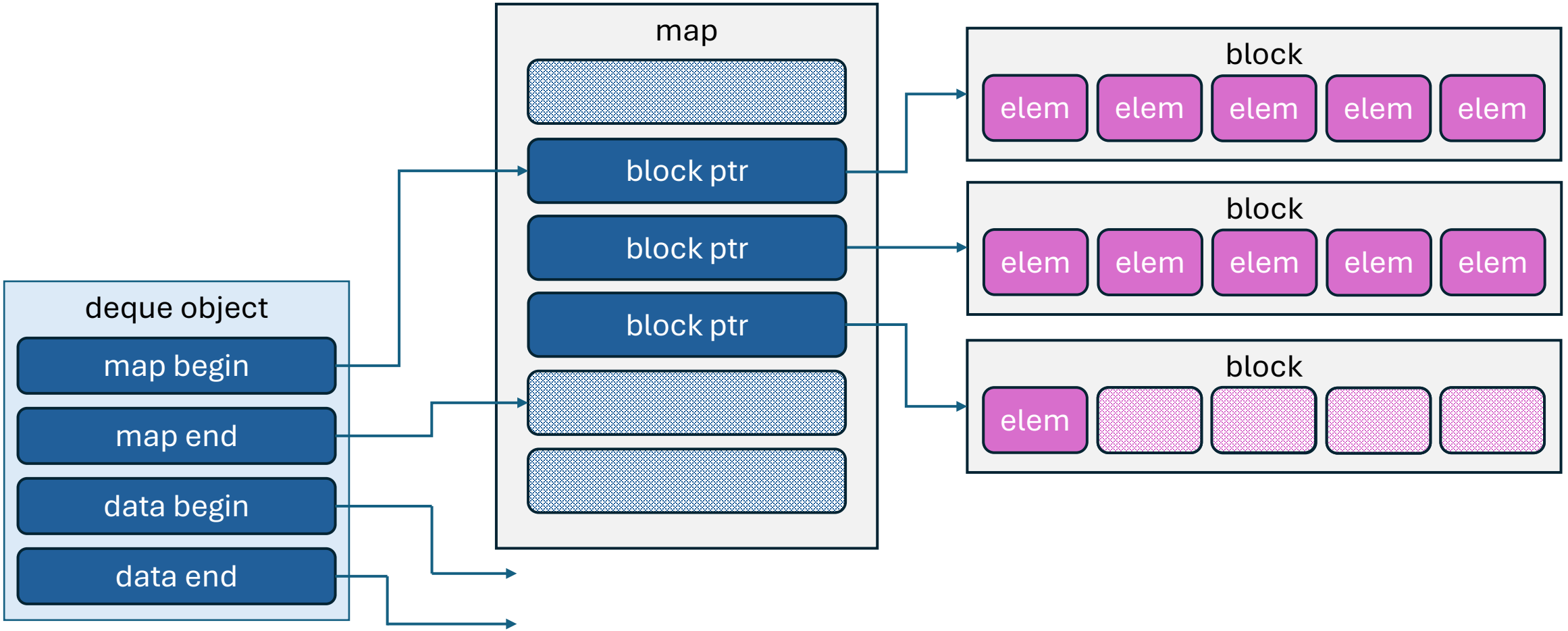
deque – First Block Created



deque – Front Block Added



deque – Back Block Added



Node-based Containers

- Each element resides in its own separate memory allocation
  - You can think of this as a block-based container with a block size of 1
  - Which provides very smooth linear memory growth
  - But makes for high overhead for small elements
- Locality *and* predictability of reference are terrible
  - Every element is somewhere random in main memory
  - So iteration performance is poor
  - If possible, load node-based containers in one go to improve locality
- But for large elements, these factors are not important
  - The overhead is 2 pointers/element—not a problem for big objects
  - If your elements are big, fetching a page for every element may not be inefficient

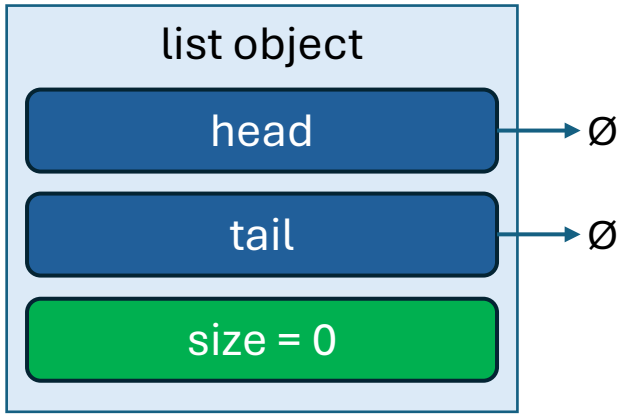
Node-based Containers

- Optimal stability
  - No existing elements are ever moved
  - So links are never invalidated
- Objects which cannot be copied *or* moved are well supported
  - Emplace-construct and splice if needed
  - You can’t do this with a **vector** or **deque**
- **list** and **forward_list**
  - The node-based sequence containers
- **map**, **set**, etc…
  - The (ordered) associative containers
- **unordered_map**, **unordered_set**, etc…
  - The unordered associative containers

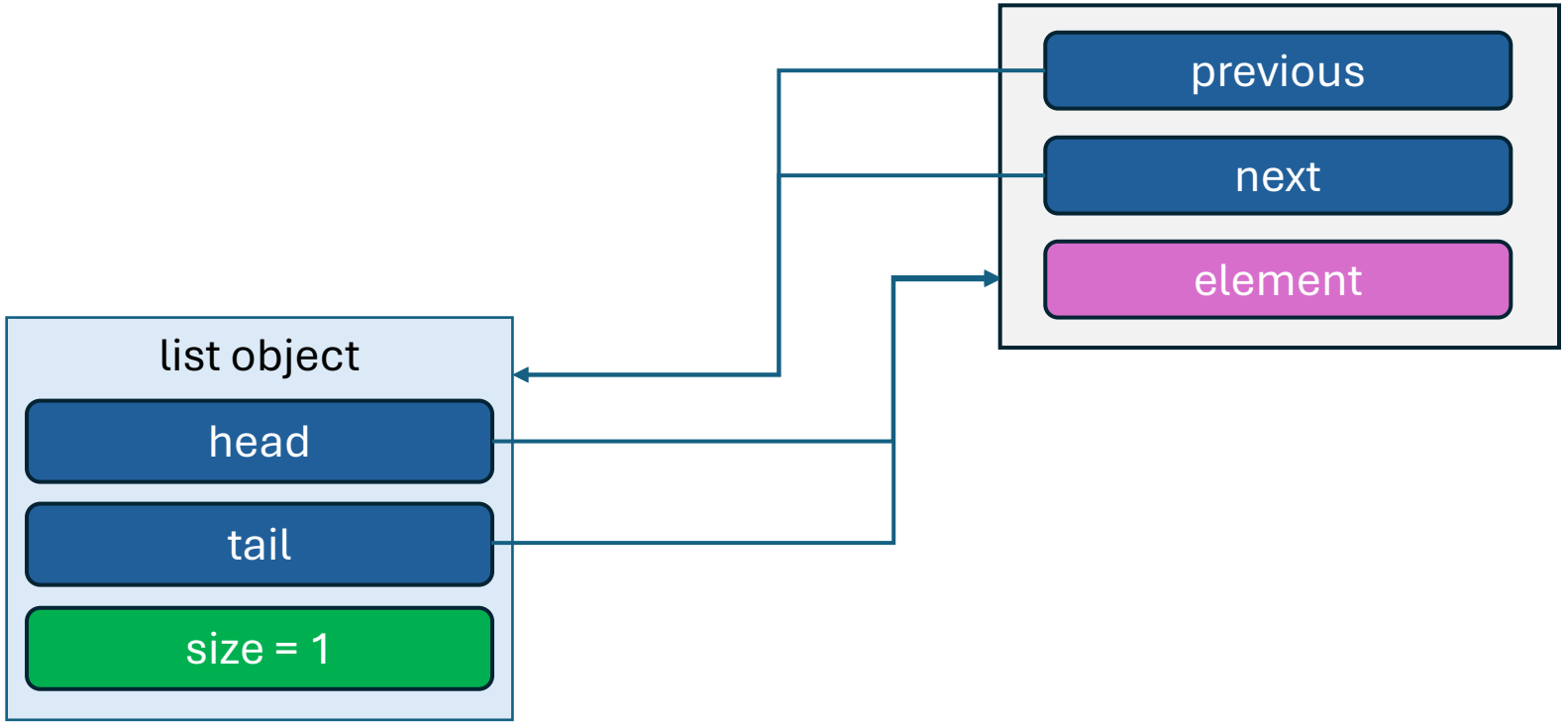
list – Features

- The SL names for the linked list containers are slightly confusing
  - **list** is the double-linked list
  - **forward_list** is the single-linked list (what most people call a “linked list”)
- Guaranteed *O*(1) insertion and deletion anywhere
- Offers *O*(1) splicing of elements already in the container
  - But *O*(*n*) splicing of elements from another container
- Offers fast sorting (without moving elements)
  - Only the pointers are swapped around
- Maintains a size field because the **size** function is specified to be *O*(1)
  - This might be an example of overspecified container requirements
  - What if I want a double-linked list but don’t want to pay for the size field?
  - What if I want *O*(1) splicing from another container?

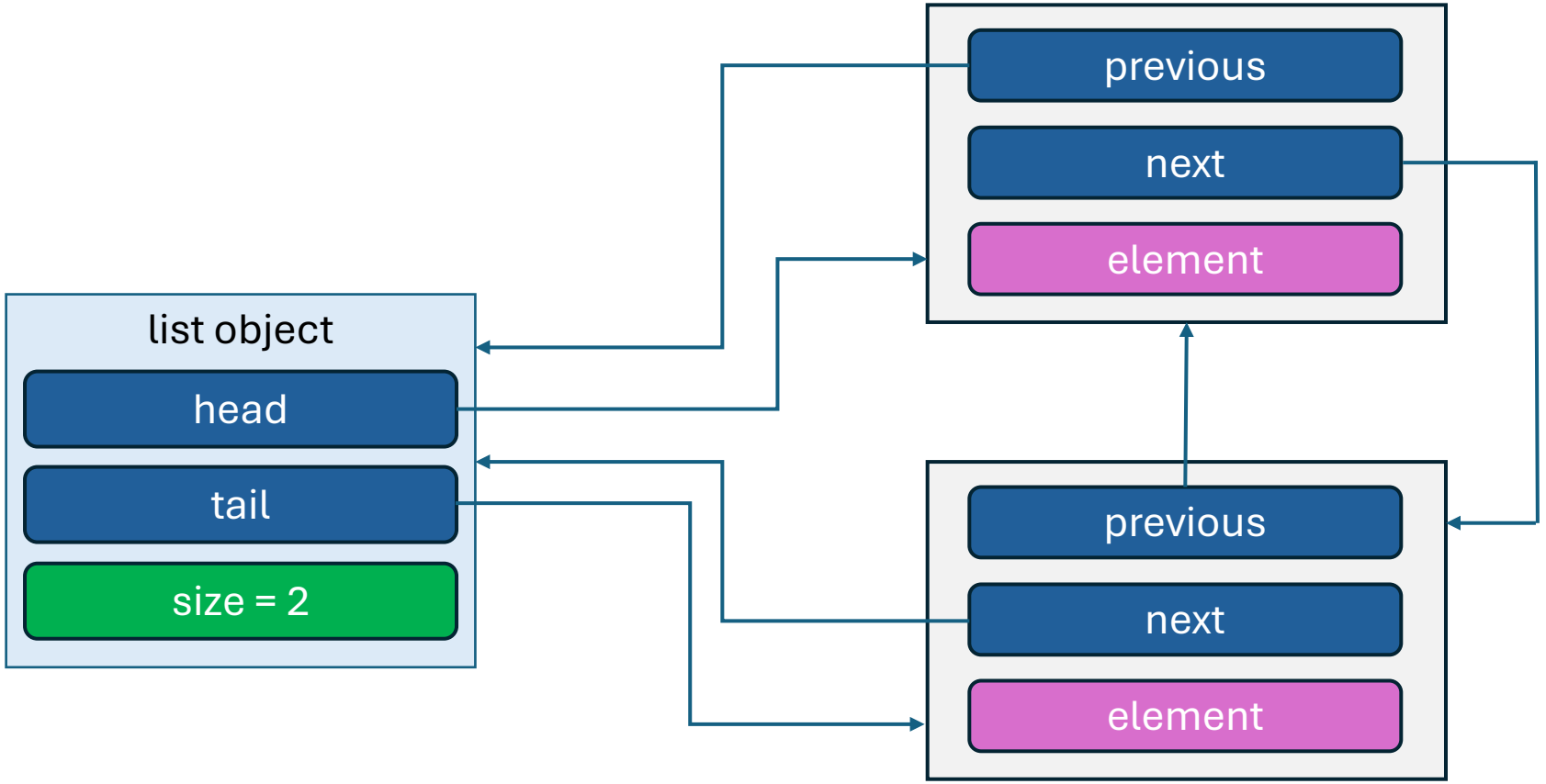
list – Default Constructed



list – One Element



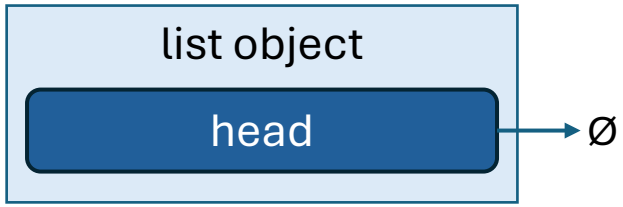
list – Two Elements



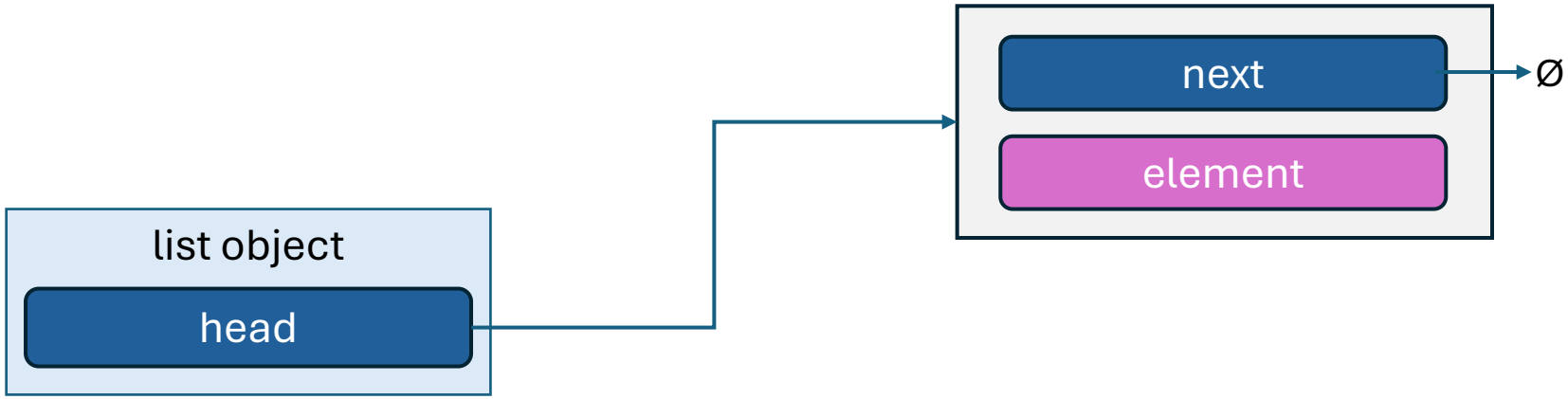
forward_list

- **forward_list** is very much like list, but with only a next pointer
  - A classic linked list, equivalent to “a hand-written C-style singly linked list”
- You lose some functionality:
  - No **size** function—you have to iterate and count the elements yourself
  - No back operations
- But you get a smaller space overhead
  - One pointer per element and the container object is one pointer
- Useful if:
  - You only care about forward traversals
  - You don’t mind putting things on the front
  - You don’t need to know the size
  - You care a lot about space overhead

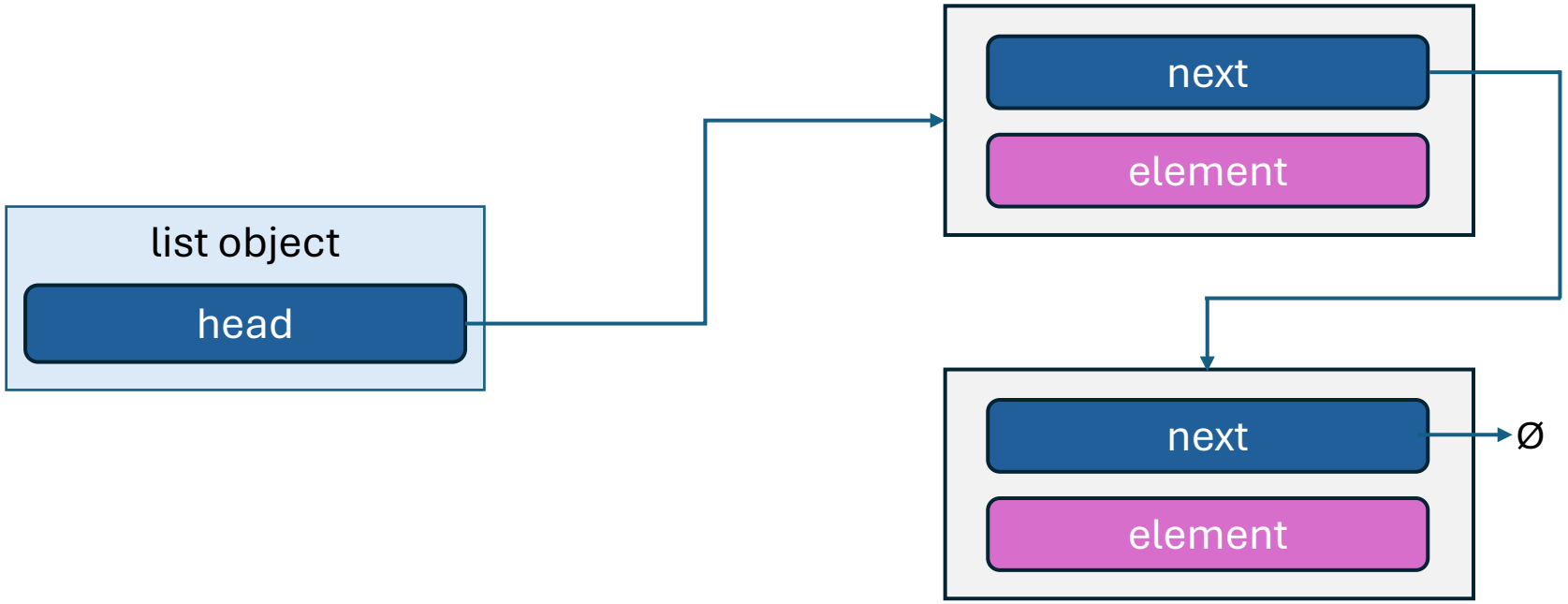
forward_list – Default Constructed



forward_list – One Element



forward_list – Two Elements



Logging Example – The Problem

- The railroad simulator needed to log a great many small records
  - To save the results of the simulation at every time step
  - This needed to happen without slowing things down
  - These logs always grew from the back
  - They needed to be iterated very quickly to display many reports at once
  - The user controlled the available memory and the length of the simulation
  - So the ability to fill up memory was a requirement
- How should I store these logs?
  - A **vector** is very fast to iterate, but won't meet the growth requirements
  - A **list** fills up memory smoothly, but is very expensive of time and space
  - A **deque** doesn't appear to fit the problem: we don't need a double-ended queue

Logging Example – The Solution

- Use a **deque**
  - Fast to insert at the back
  - Fast to iterate
  - Data are held in blocks so good locality of reference
  - Fills memory well since the block size doesn't grow (as it does with **vector**)
- Which is what I did
  - And it worked well enough
  - But—there was a gotcha I didn’t know about
  - I was leaving performance on the table

Logging Example – The Surprise

- I recently discovered that the MSVC library has a very strange **deque**
  - Specifically for my case, the block size was effectively 1 element
- This meant I was actually using an array of pointers
  - A node-based container, not a block-based container
  - I might as well have used a linked list
- I ran a little test to explore this:
  - Comparing filling a **deque** with small, simple structures (4 **long long**s, 32 bytes)
  - To filling a **vector** of **vector**s with the same structure (using good reserve sizes)
    - This was a stand-in for a deque with a better block size
  - The **vector** of **vector**s was over 4 times faster
  - I also tried a **list** and it was only slightly slower than a **deque**
- Check your **deque** implementation before you count on it (see Links)

Container Adaptors

- Container adaptors are wrappers around containers
  - They have one or more template arguments specifying the underlying container type(s)
  - They provide a particular API and specialized behavior (e.g. a stack interface)
- Adaptors maintain their own container
  - They do not reference your container
  - If you pass your container to the constructor, the adaptor will copy or move it
  - Moving something like a vector into an adaptor gives you control of the initial state
- Adaptors have specific requirements of the container they wrap
  - Various features will not compile if you don’t meet those requirements
  - These requirements limit which containers can be used

Sequence Container Adaptors

- **stack**
  - Implements a stack (LIFO)
  - Works with **vector**, **list** and **deque**
  - Defaults to **deque**
- **queue**
  - Implements a queue (FIFO)
  - Works with **list** and **deque** (but not **vector**)
  - Defaults to **deque**
- **priority_queue**
  - Implements a priority queue
  - Typically implemented as a heap data structure internally
  - Works with **vector** and **deque**
  - Defaults to **vector**

Adaptor Default Quirk

- **stack**
  - You might have noticed that **stack** defaults to **deque** – why?
  - The only reason to use **deque** for a stack would be if you need linear growth behavior
    - So it would have to be a (potentially) very large stack
  - For most uses **vector** would surely be a more appropriate container
  - But people tend to use defaults! Wouldn't you?

```
stack<int> my_stack;                  // This uses a deque!
stack<int, vector<int>> my_stack;     // This uses a vector.
```

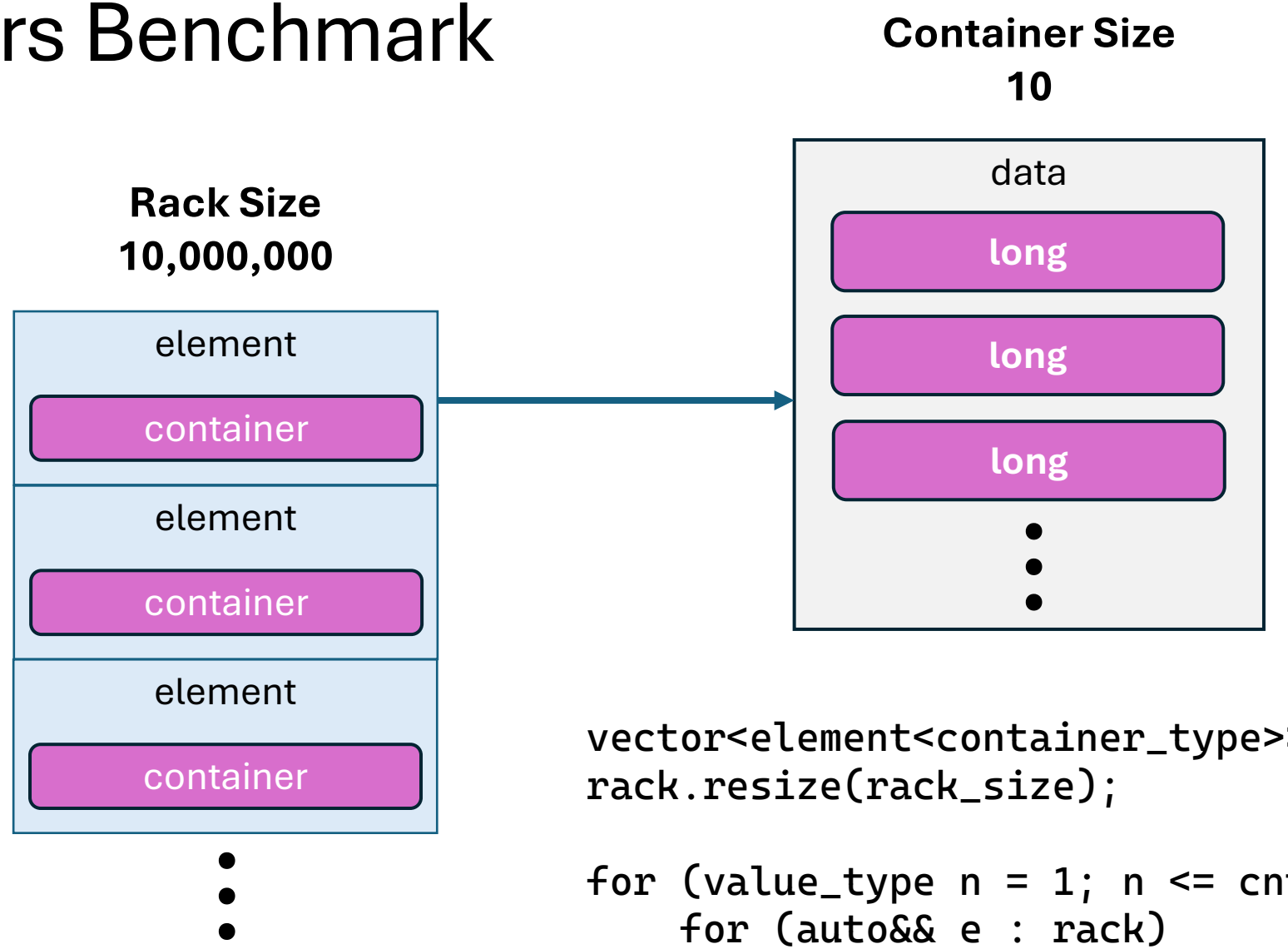
Aging STL Design Guidance

- The SL inherits a lot of design guidance from the original STL
  - And that's mostly a good thing!
  - But some of the messages are based on 30-year-old assumptions about hardware
- **vector** is the go-to sequence container, for good reasons
  - But vector has no **push_front** / **pop_front**
  - So you are led to think that front operations on a **vector** are a bad idea
  - You can't use **queue** on **vector** because it requires front operations
  - And this is deliberate—that is exactly the message that the SL intends to send
- But for small sizes, **deque** is a terrible choice
  - Huge overhead for the "map" (block lookup table)
  - Potentially huge overhead for extra space in the block if a reasonable block size is used
  - Node-based performance if the block size is too small
- And **list** is node-based, so it's even worse

Aging STL Design Guidance

- But for small sizes, a **vector** ends up in the inner cache
  - So shifting elements is blazingly fast
  - It *far* out-performs **deque** or **list** for small sizes
  - Modern hardware is very different from the mid-1990's

Containers Benchmark




```
vector<element<container_type>> rack;
rack.resize(rack_size);

for (value_type n = 1; n <= cntr_size; ++n)
    for (auto&& e : rack)
        e.add(n);
```

Containers Benchmark

Container	Reserve	End	Time (ms)	Memory (MB)
list	n/a	front	1782	3581

Rack Size = 10,000,000
Container Size = 10

Microsoft Windows 11 Pro, 12th Gen Intel Core i9-12900 (2400 Mhz, 16 Cores, 24 Logical Processors)

Containers Benchmark

Container	Reserve	End	Time (ms)	Memory (MB)
list	n/a	front	1782	3581
deque	n/a	front	1512	1630

Rack Size = 10,000,000
Container Size = 10

Microsoft Windows 11 Pro, 12th Gen Intel Core i9-12900 (2400 Mhz, 16 Cores, 24 Logical Processors)

Containers Benchmark

Container	Reserve	End	Time (ms)	Memory (MB)
list	n/a	front	1782	3581
deque	n/a	front	1512	1630
vector	**none**	front	1833	1589

Rack Size = 10,000,000
Container Size = 10

Microsoft Windows 11 Pro, 12th Gen Intel Core i9-12900 (2400 Mhz, 16 Cores, 24 Logical Processors)

Containers Benchmark

Container	Reserve	End	Time (ms)	Memory (MB)
list	n/a	front	1782	3581
deque	n/a	front	1512	1630
vector	**none**	front	1833	1589
vector	10	front	607	695

Rack Size = 10,000,000
Container Size = 10

Microsoft Windows 11 Pro, 12th Gen Intel Core i9-12900 (2400 Mhz, 16 Cores, 24 Logical Processors)

Containers Benchmark

Container	Reserve	End	Time (ms)	Memory (MB)
list	n/a	front	1782	3581
deque	n/a	front	1512	1630
vector	**none**	front	1833	1589
vector	10	front	607	695
vector	10	back	561	695

Rack Size = 10,000,000
Container Size = 10

Microsoft Windows 11 Pro, 12th Gen Intel Core i9-12900 (2400 Mhz, 16 Cores, 24 Logical Processors)

Containers Benchmark

Container	Reserve	End	Time (ms)	Memory (MB)
list	n/a	front	1950	3163
deque	n/a	front	1600	695
vector	**none**	front	2566	953
vector	100	front	2838	410
vector	100	back	1038	410

Rack Size = 1,000,000
Container Size = 100

Microsoft Windows 11 Pro, 12th Gen Intel Core i9-12900 (2400 Mhz, 16 Cores, 24 Logical Processors)

Associative Containers

- Maps or sets
  - Maps associate a key with a value
  - Sets simply store a key
- Unique or multiple value
  - Non-multiple containers have one value per key (**map**, **set**)
  - Multiple containers support more than one value per key (**multimap**, **multiset**)
- Ordered
  - Maintain keys in order using a user-provided comparitor (**map**, etc.)
  - Have $O(\log n)$ lookup
- Unordered
  - What everyone else calls hash tables (**unordered_map**, etc.)
  - Have $O(1)$ lookup on average, $O(n)$ worst case
  - Are node-based to meet the invalidation requirements

Associative Container Details

- **map** and **set** are the same thing
  - They are typically implemented with exactly the same code
  - Except **map** stores a {key, value} **pair**, while **set** stores only a key
- The **multi** versions are also the same code
  - The only difference is that key collisions are allowed to be inserted
- The ordered versions are typically implemented as a search tree
- The unordered versions have a lookup table indexed by a hash function
- All of them support splicing and re-keying and other node manipulations
  - More about this in Part 2

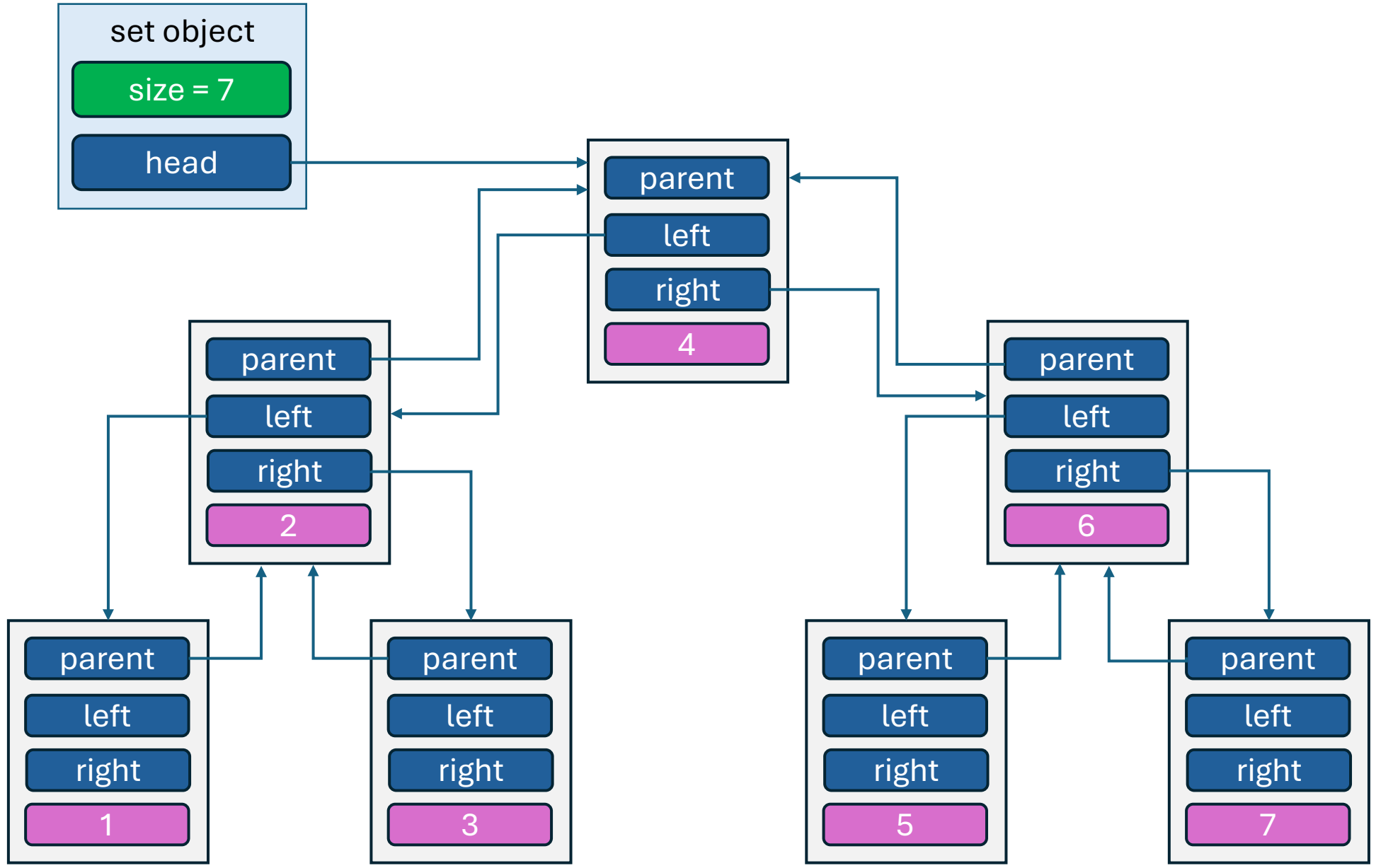
set

- A set stores elements in a user-specified order…
- So when I want an ordered list, use a set, right?
- Probably, if one or more of these is true:
  - You have an unknown number of large elements
  - Elements are frequently being added or removed
  - You need stability (elements don’t move around in memory)
- But not if these are true
  - Your list is short or you know in advance how big it is
  - You aren’t going to be changing the list (much)
  - You don’t care about stability
- Instead, use **vector** and sort it
  - You can then do $O(\log n)$ lookup with the binary search algorithm
  - For small counts of small, moveable elements, this will be *much* faster

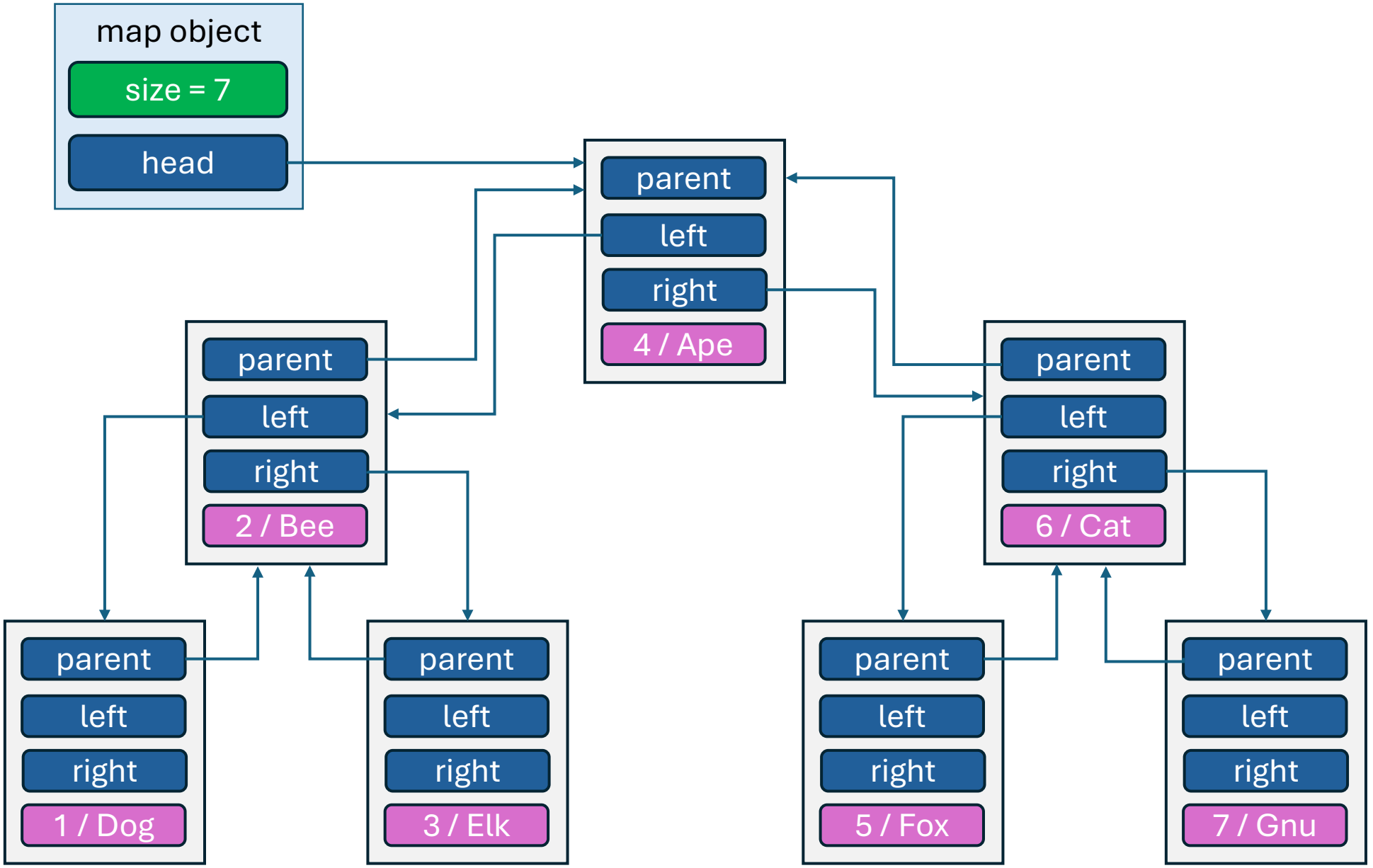
map

- A map stores elements in a user-specified order
- And provides an association between the key and a value
  - This is often called a dictionary or associative array
- It provides an easy way to get $O(\log n)$ lookup and ordering
- And it offers $O(\log n)$ insertion of new elements
- You may get better performance by using contiguous memory storage
  - For the same conditions as with sets

set


set object
size = 7
head
parent
left
right
4
parent
left
right
2
parent
left
right
6
parent
left
right
1
parent
left
right
3
parent
left
right
5
parent
left
right
7

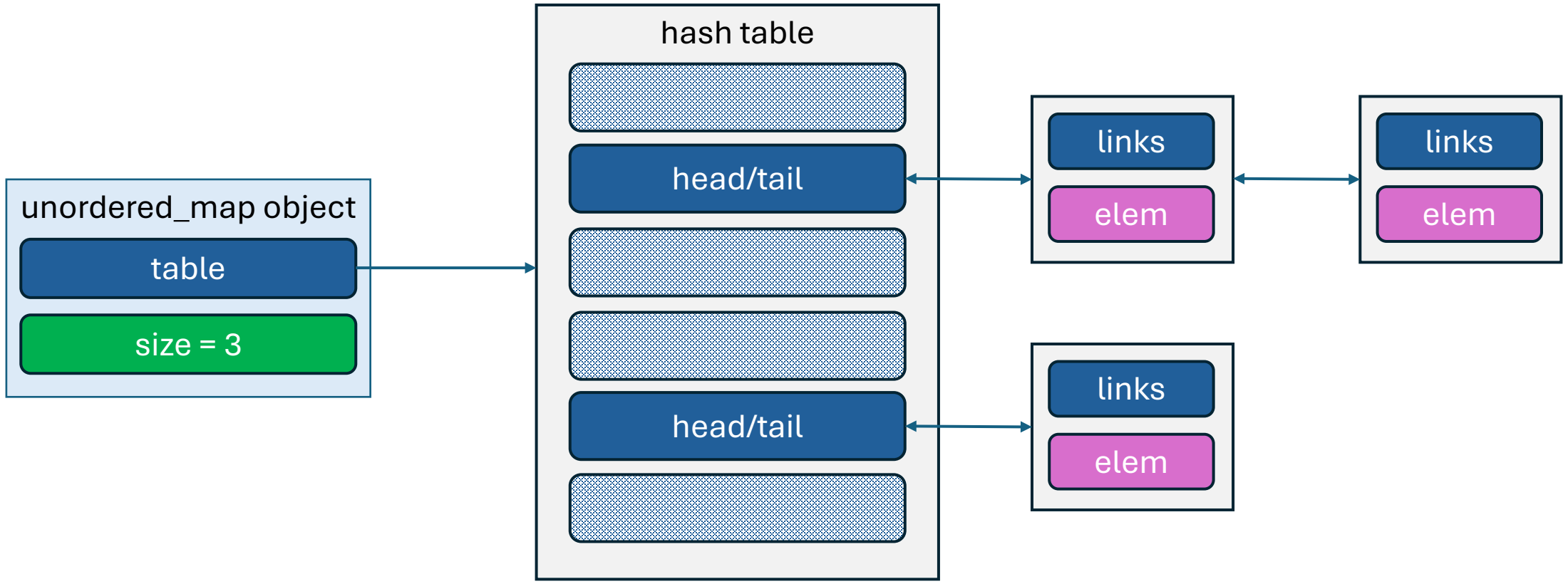
map


map object
size = 7
head
parent
left
right
4 / Ape
parent
left
right
2 / Bee
parent
left
right
6 / Cat
parent
left
right
1 / Dog
parent
left
right
3 / Elk
parent
left
right
5 / Fox
parent
left
right
7 / Gnu

unordered_map (etc.)

- **unordered_map** has the same functionality as **map**
- All the other variations (**_set**, **_multiset**, **_multimap**) are available
- Implemented with a hash table (rather than a search tree)
  - Entries in the table are the heads of double-linked lists of element nodes
- A hash table can provide much better performance than a search tree
  - Lookup is roughly *O*(1) if the hash function is good (i.e. few collisions)
- The SL unordered containers are known to lag behind the state of the art
  - Hash tables are an area of ongoing research
  - These were standardized in C++11, and it's hard to change C++
  - The API specification has constrained these to be node-base
  - Nonetheless, they are still pretty good
- Kevin Carpenter's advice is to reach for these before the tree-based versions

unordered_map (etc.)



Links

- Related talks
  - Timur Doumler: *Want fast C++? Know your hardware!* (CppCon 2016)
    www.youtube.com/watch?v=BP6NxVxDQIs
  - Fedor Pikus: *Unlocking Modern CPU Power - Next-Gen C++ Optimization Techniques* (C++Now 2024)
    www.youtube.com/watch?v=wGSSUSeaLgA
  - Mathieu Ropert: *Heaps Don't Lie, Guidelines for Memory Allocation in C++* (ACCU 2025)
    www.youtube.com/watch?v=74WOvgGsyxs
  - Jon Kalb: *Alex Stepanov, Generic Programming, and the STL* (ACCU 2025)
    www.youtube.com/watch?v=d_DlHYX5w5w
  - Alan Talbot: *Moving Faster: Everyday efficiency in modern C++* (CppCon 2018)
    www.youtube.com/watch?v=EovBkh9wDnM
  - Alan Talbot: *How to Choose the Right Standard Library Container* (CppCon 2019)
    www.youtube.com/watch?v=yjPKVOYcw28
  - Alan Talbot: *New (& Old) C++ Standard Library Containers - How to Choose the Right Container in C++26 and Beyond* (ACCU 2025)
    www.youtube.com/watch?v=TtbYGico7bI

Links

- **deque**
  - Raymond Chen: *Inside STL: The deque, design*
    devblogs.microsoft.com/oldnewthing/20230809-00/?p=108577
  - Raymond Chen: *Inside STL: The deque, implementation*
    devblogs.microsoft.com/oldnewthing/20230810-00/?p=108587
- Benchmark
  - **containers-benchmark** – The code I used to get the container comparison numbers
    github.com/talbotcpp/containers-benchmark